

게임엔진 기초

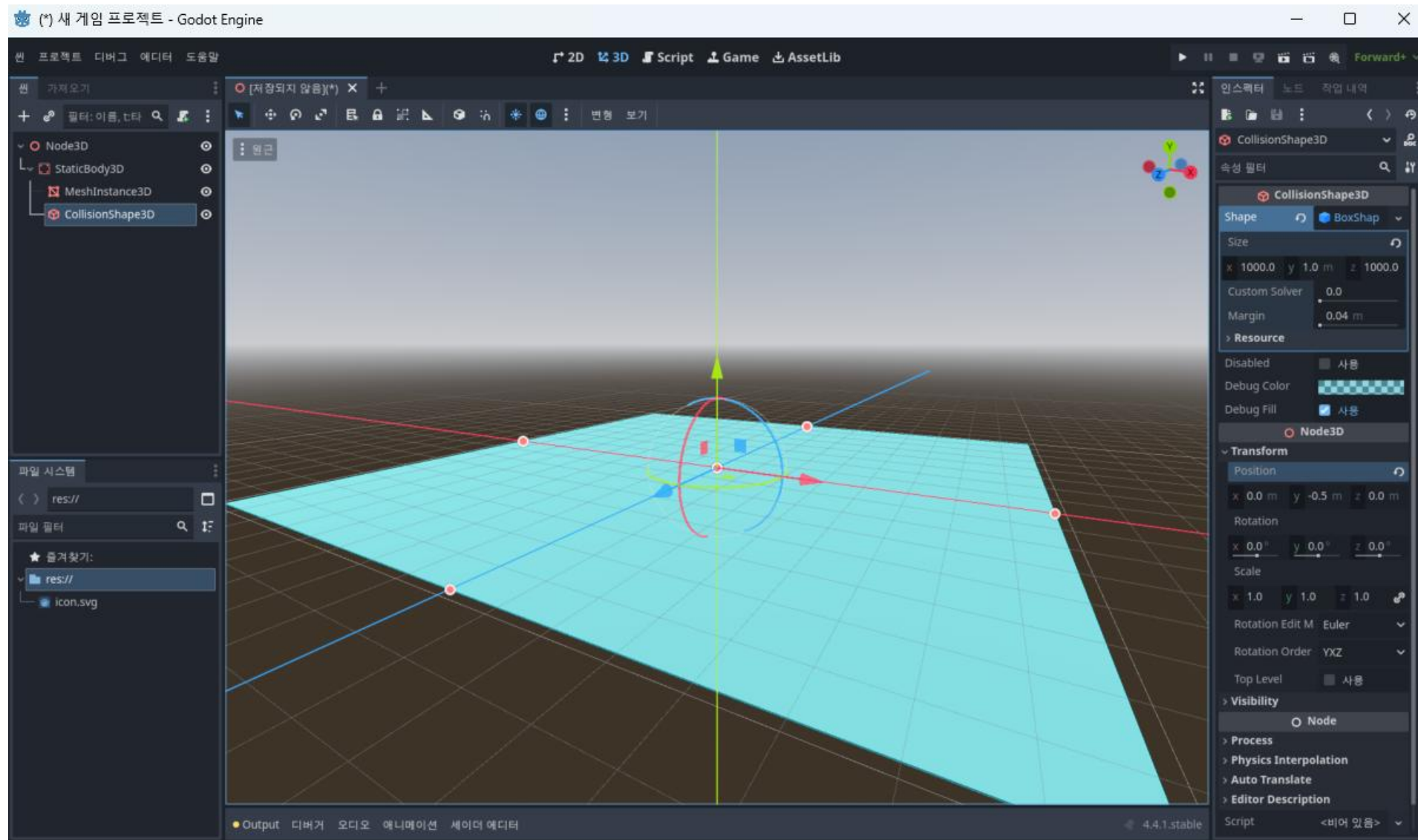
Godot Physics Game

강영민

동명대학교 게임그래픽학과



게임 프로젝트 만들기



프로젝트 만들기

1. 프로젝트 설정

- 새 프로젝트 생성.

- 기본 장면(Scene)을 3D로 설정: Spatial 노드를 루트로.

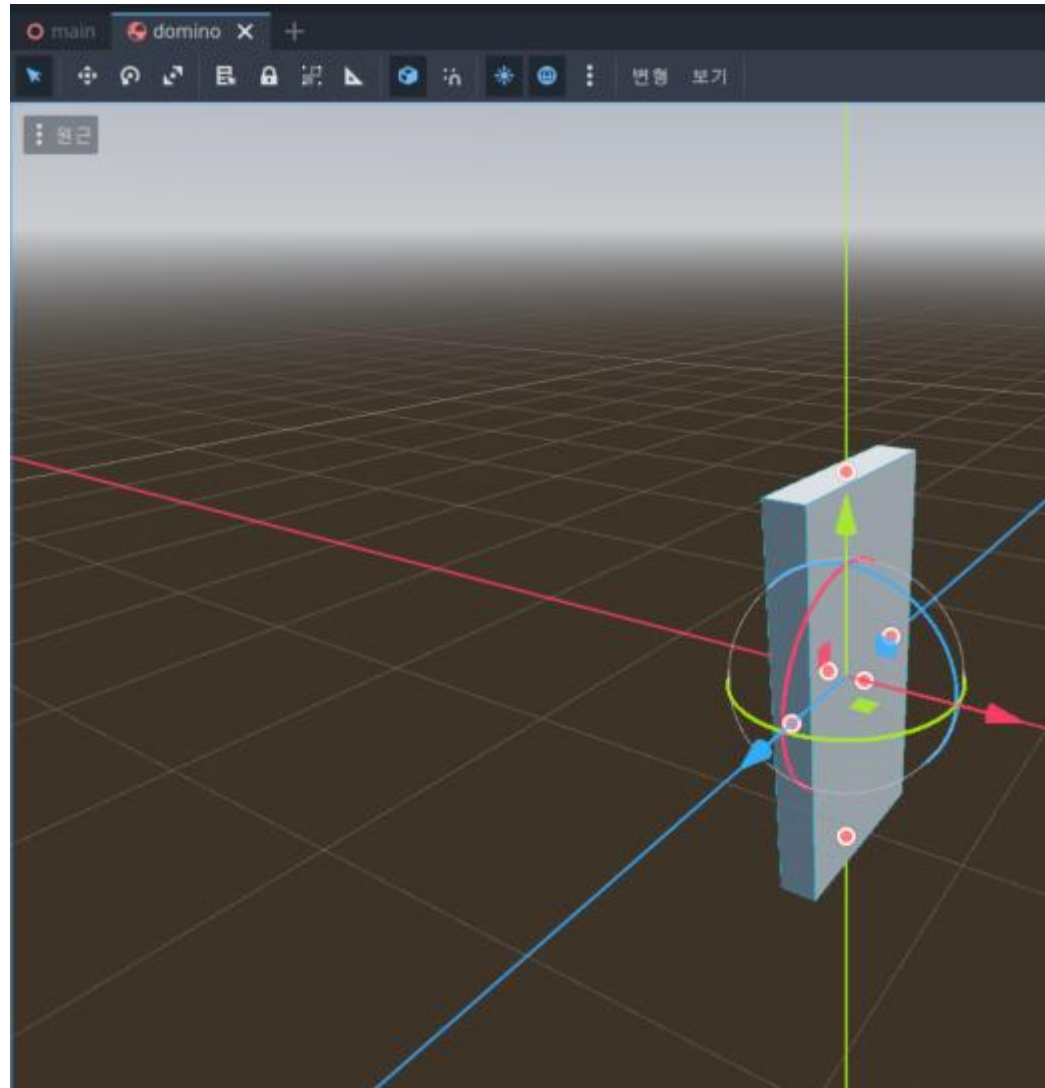
- 장면 이름: Main.tscn.

- 필요한 노드 추가:

- Floor: StaticBody3D + CollisionShape3D (BoxShape로 바닥 만들기) + MeshInstance3D (PlaneMesh로 시각화).
- Camera: Camera3D (초기 위치: (0, 5, 10), 원점 바라보게).
- DirectionalLight3D: 조명 추가 (위치: (0, 10, 0)).
- UI: Control 노드 추가 (CanvasLayer로). 안에 LineEdit (도미노 개수 입력), Button (생성 버튼), Label (안내 텍스트).

도미노 모델: 간단히 RigidBody3D + MeshInstance3D (BoxMesh: 크기 0.2x2x1) + CollisionShape3D (BoxShape). 이것 Domino.tscn으로 저장해서 prefab처럼 사용.

도미노



간단한 UI 구현 – main.gd → main.tscn에 붙이기

```
extends Node3D
```

```
@onready var domino_count_input: LineEdit = $CanvasLayer/UIContainer/LineEdit
```

```
@onready var generate_button: Button = $CanvasLayer/UIContainer/GenerateButton
```

```
@onready var camera: Camera3D = $Camera3D
```

```
var domino_scene: PackedScene = preload("res://Domino.tscn")
```

```
var dominoes: Array = []
```

```
func _ready():
```

```
    generate_button.connect("pressed", _on_generate_pressed)
```

```
func _on_generate_pressed():
```

```
    var count = int(domino_count_input.text)
```

```
    if count > 0:
```

```
        generate_spiral_dominoes(count)
```

도미노 생성 – main.gd에 추가

```
func generate_spiral_dominoes(num_dominoes: int):  
    # 기존 도미노 제거  
    for d in dominoes:  
        d.queue_free()  
    dominoes.clear()  
  
    var a: float = 0.1 # 시작 반경  
    var b: float = 0.1 # 반경 증가율 (도미노 크기에 맞게 조정)  
    var theta: float = 0.0  
    var theta_step: float = PI / 6 # 약 30도 간격, 도미노가 서로 닿을 정도로 조정  
  
    for i in range(num_dominoes):  
        var r = a + b * theta  
        var x = r * cos(theta)  
        var z = r * sin(theta)  
  
        var domino = domino_scene.instantiate()  
        domino.position = Vector3(x, 1.0, z) # y=1.0으로 서 있게 (도미노 높이 절반)  
        domino.rotation = Vector3(0, theta + PI/2, 0) # 나선 방향으로 회전 (앞 도미노를 향하게)  
        add_child(domino)  
        dominoes.append(domino)  
  
        theta += theta_step
```

오류: UI 없음

```
@onready var domino_count_input: LineEdit = $CanvasLayer/UIContainer/LineEdit  
@onready var generate_button: Button = $CanvasLayer/UIContainer/GenerateButton
```


UI 만들기

UI 루트 노드 만들기

- 1.Main.tscn 열기
- 2.Scene 탭에서 루트 노드(보통 Node3D) 우클릭 → Add Child Node
- 3.검색창에 CanvasLayer 입력 → 선택 → Create → 이름은 그대로 CanvasLayer 또는 UI로 바꾸기
- 4.CanvasLayer 선택된 상태에서 다시 우클릭 → Add Child Node
- 5.검색창에 Control 입력 → 선택 → Create → 이름은 UIContainer 정도로 바꾸기
- 6.이제부터 모든 UI는 이 Control 안에 넣을 것

라인에디트

LineEdit과 Button 추가하기 (드래그로 위치 조정)
CanvasLayer → UIContainer 선택된 상태에서

1. LineEdit 만들기

1. 우클릭 → Add Child Node → LineEdit 검색 → Create
2. 이름: LineEdit (또는 DominoCountInput)
3. Inspector에서
 1. Text: 100 (기본값)
 2. Placeholder Text: 도미노 개수 입력...
 3. Rect → Size: 300 x 60 정도
 4. Anchor: Top Wide (상단 전체 너비)
 5. Margin → Left: 50, Top: 50, Right: -50 (화면 양옆 여백)

버튼

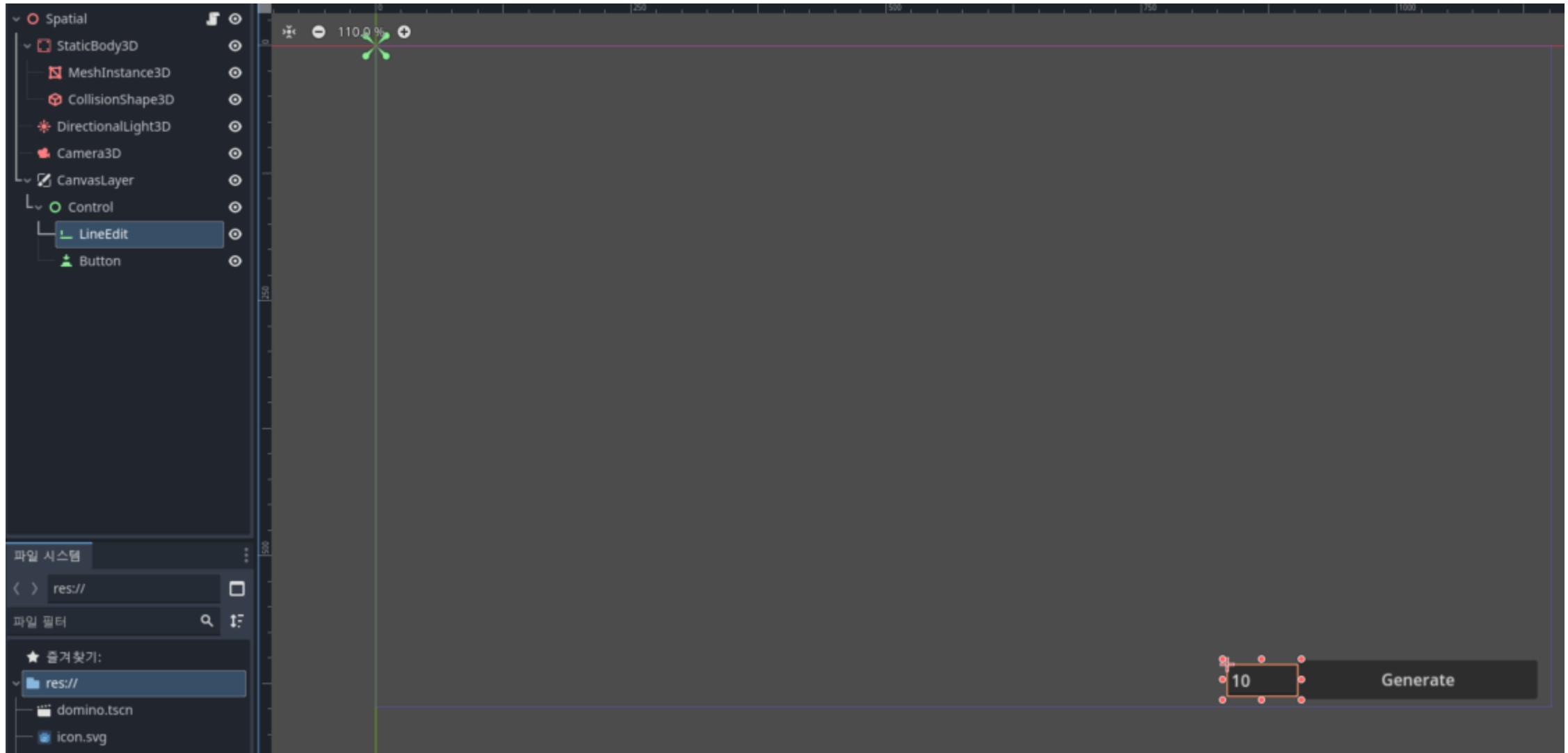
Button 만들기

UIContainer 우클릭 → Add Child Node → Button 검색 → Create

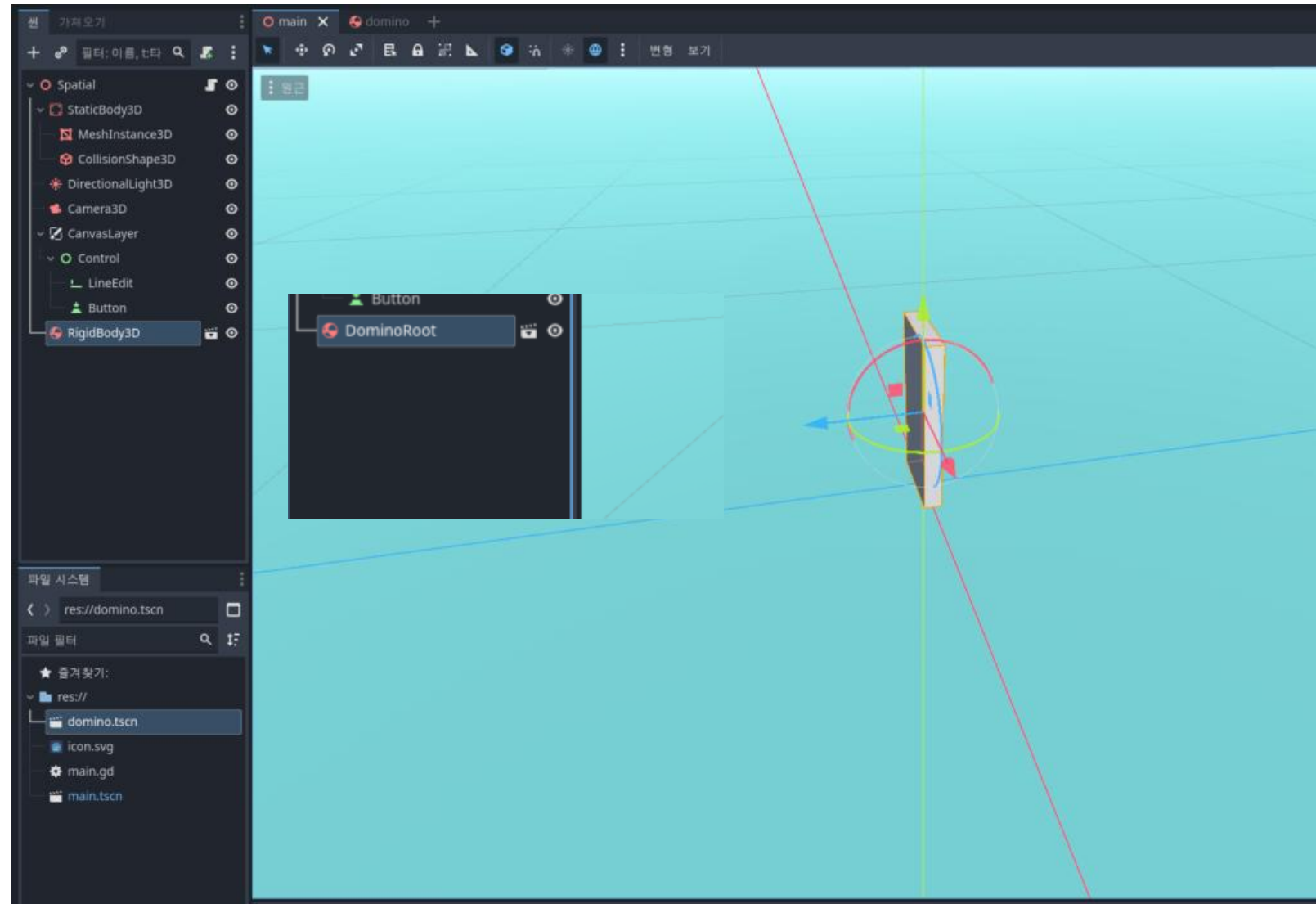
•Inspector에서

- Text: Generate 도미노! (또는 그냥 Generate)
- Rect → Size: 300 x 80
- Anchor: Top Wide
- Margin → Left: 50, Top: 130 (LineEdit 바로 아래로)

UI 배치



새로운 방법 – 도미노 루트 설치



Generation 고민하기

```
func generate_spiral_dominoes(num_dominoes: int):  
    # 기존 제거  
    for d in dominoes:  
        d.queue_free()  
    dominoes.clear()  
  
    # 첫 번째 도미노는 그냥 원점에  
    var previous_domino: Node3D = null  
  
    # 나선 파라미터 (마음껏 조절 가능)  
    const FORWARD_STEP: float = 0.95 # 한 칸 앞으로  
    var turn_angle: float = deg_to_rad(-8.0) # 한 번에 몇 도씩 꺾을지 (음수 = 시계방향)  
    const ANGLE_DECAY = 0.99
```

Generation 고민하기

```
for i in range(num_dominoes):
    var domino = domino_scene.instantiate() as Node3D

    if i == 0:
        # 첫 번째 도미노는 그냥 원점에 세운다
        domino.position = Vector3(FORWARD_STEP, 0, 0)
        domino_root.add_child(domino)
    else:
        # 이전 도미노를 부모로 해서 local transform 적용
        previous_domino.add_child(domino)

        # 앞으로 조금 이동 (local Z축)
        domino.transform = domino.transform.translated(Vector3(FORWARD_STEP, 0, 0))

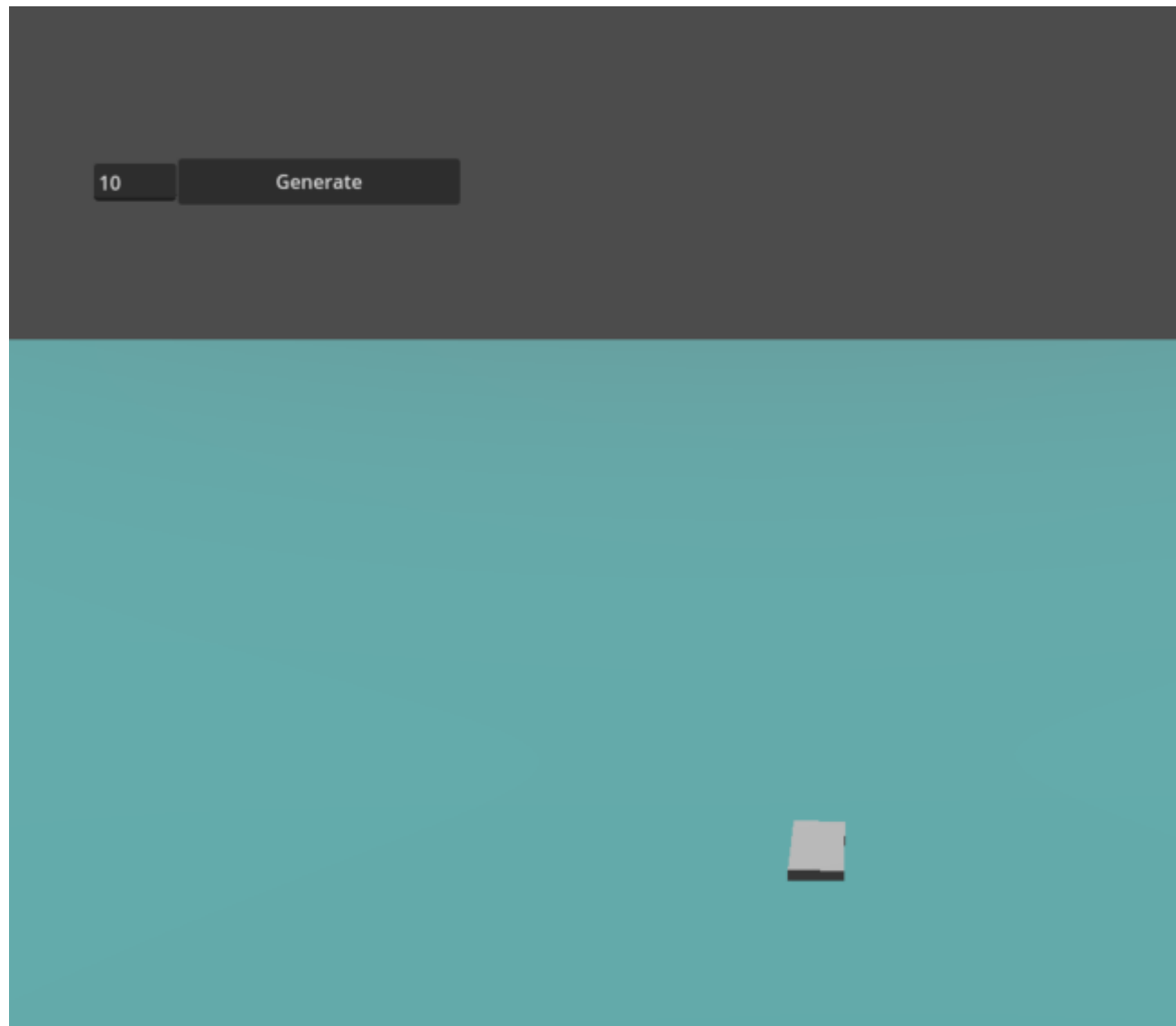
        # 살짝 회전 (Y축 기준, local)
        domino.rotate_y(turn_angle)

    dominoes.append(domino)
    previous_domino = domino # 다음 루프를 위해 기억
    turn_angle *= ANGLE_DECAY # 회전을 줄여감
```

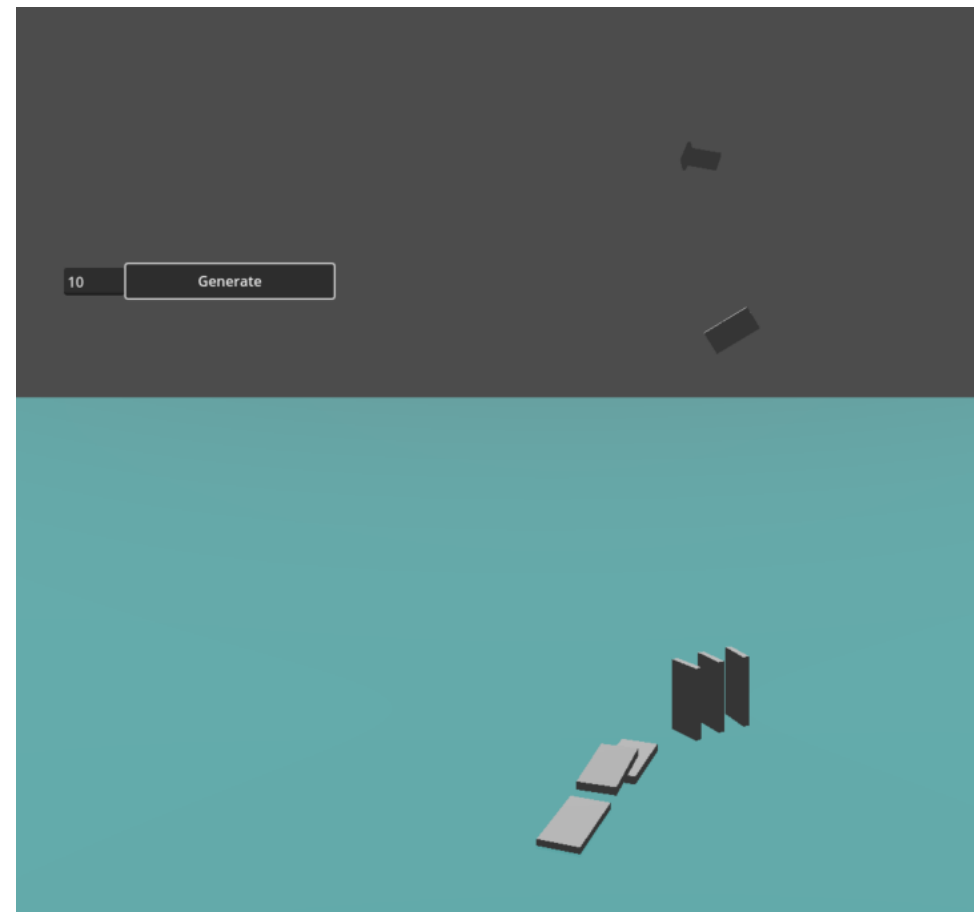
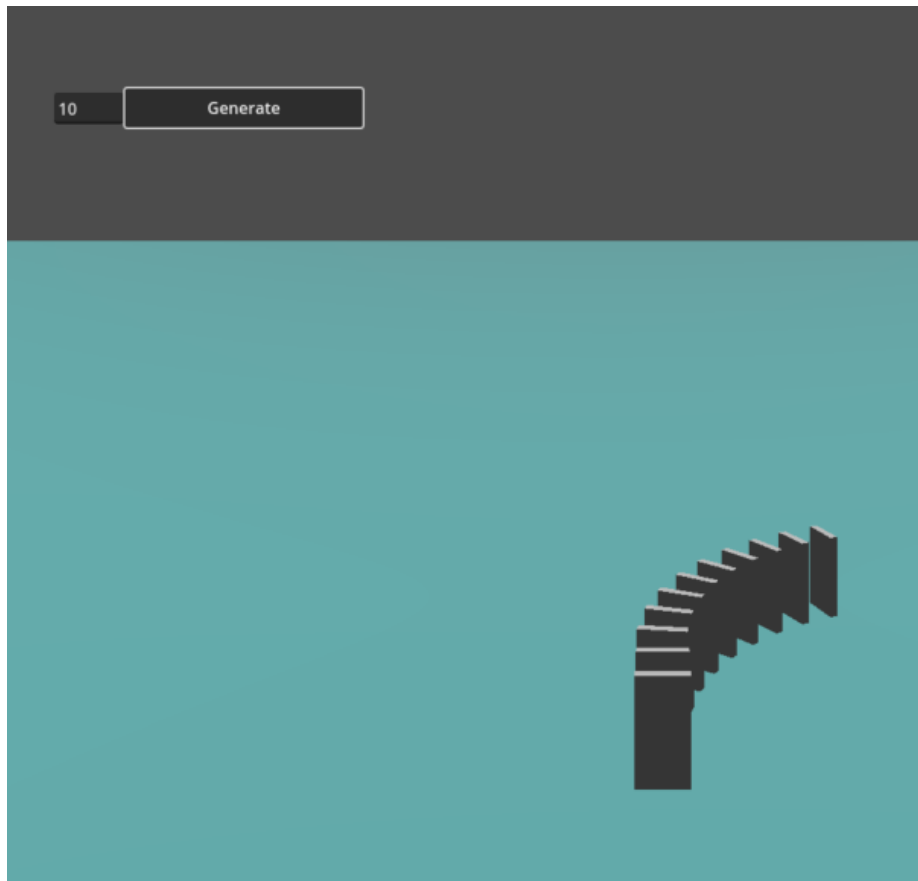
마우스 입력으로 도미노 넘기기

```
func _input(event):  
    if event is InputEventMouseButton and event.button_index == MOUSE_BUTTON_LEFT and event.pressed:  
        var mouse_pos = event.position  
        var ray_from = camera.project_ray_origin(mouse_pos)  
        var ray_to = ray_from + camera.project_ray_normal(mouse_pos) * 1000  
  
        var space_state = get_world_3d().direct_space_state  
        var query = PhysicsRayQueryParameters3D.create(ray_from, ray_to)  
        var result = space_state.intersect_ray(query)  
  
        if result and result.collider is RigidBody3D:  
            var domino = result.collider  
            var force_dir = (domino.position - camera.position).normalized() # 카메라 방향으로 밀기  
            domino.apply_central_impulse(force_dir * 10) # 힘 세기 조정  
            start_camera_follow() # 카메라 따라가기 시작 (이건 우선 끄고 해야 동작)
```


넘어간다!



그러나



코드를 고쳐보자

```
@onready var domino_root: Node3D = $DominoRoot # 그냥 도미노들 담는 빈 폴더 역할만!
```

```
func generate_spiral_dominoes(num_dominoes: int):
```

```
    # 기존 제거
```

```
    for d in dominoes:
```

```
        d.queue_free()
```

```
    dominoes.clear()
```

```
    # 파라미터
```

```
    const FORWARD: float = 1.0      # 도미노 간격
```

```
    var turn: float = deg_to_rad(-9.0) # 처음엔 세게 꺾고
```

```
    const DECAY: float = 0.98        # 점점 덜 꺾이게 (자연스러운 나선)
```

```
    var current_transform := Transform3D() # 처음은 원점
```

```
    current_transform.origin = Vector3(FORWARD, 0, 0) # 첫 도미노 위치
```

코드를 고쳐보자

@

```
for i in range(num_dominoes):
    var domino = domino_scene.instantiate() as RigidBody3D

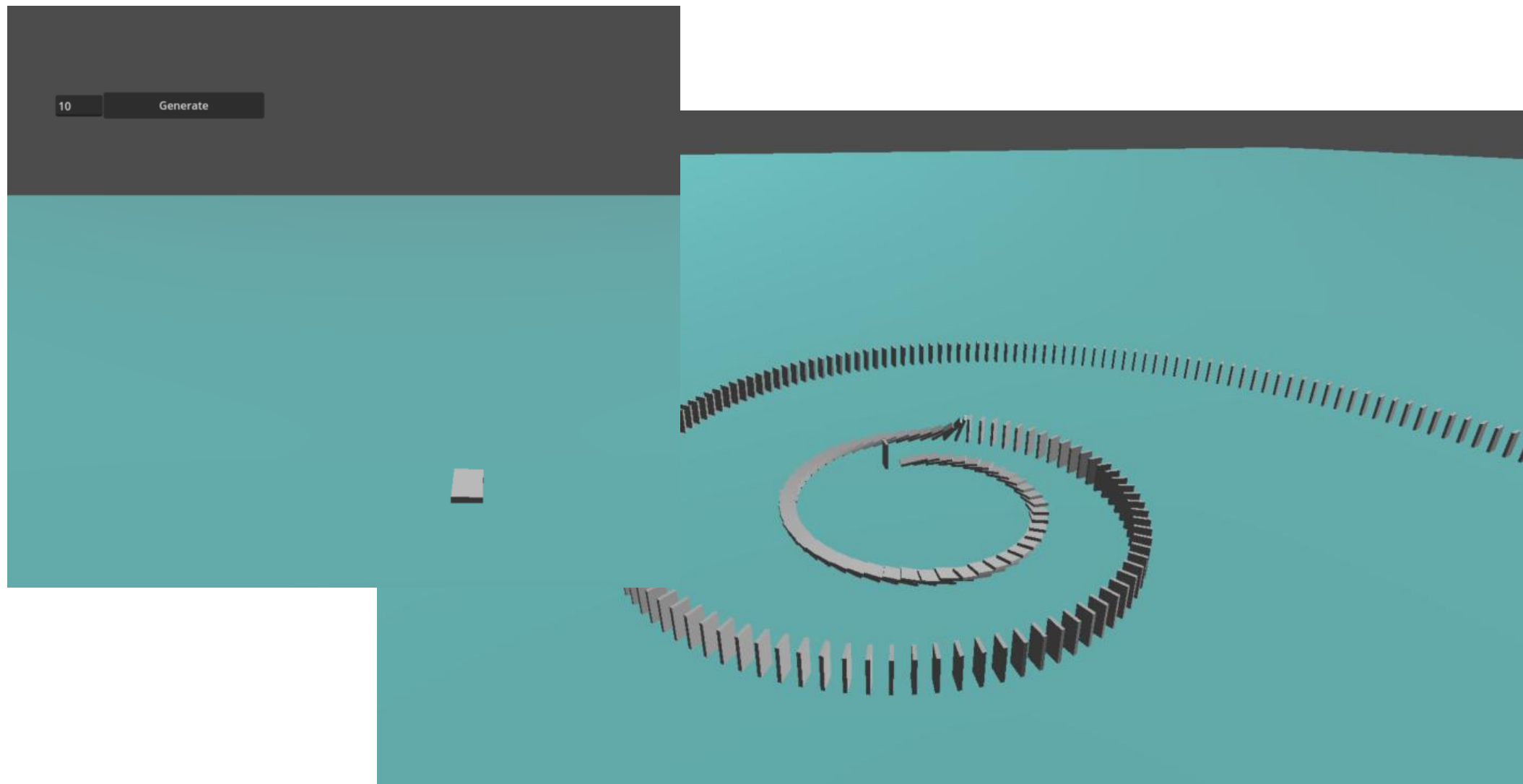
    # transform으로 위치/회전 잡고 월드에 배치
    domino.global_transform = current_transform

    domino_root.add_child(domino)
    dominoes.append(domino)

    current_transform = Transform3D(current_transform.basis, current_transform.origin) \
    * Transform3D(Basis(), Vector3(FORWARD, 0, 0)) \
    * Transform3D(Basis().rotated(Vector3.UP, turn), Vector3.ZERO)

    turn *= DECAY
```

넘어간다! 루트만 조심!



카메라 조작

```
var cinematic_mode := false
var cam_radius := 3.0      # 시작 반경
var cam_angle := 0.0
var cam_height := 5.0

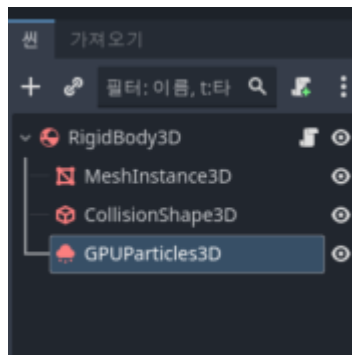
@export var radius_speed := 8.0  # 초당 얼마나 멀어질지
@export var rotate_speed := 0.35 # 회전 속도
@export var height_speed := 3.5  # 초당 얼마나 올라갈지

func start_camera_follow():
    cinematic_mode = true
    cam_radius = 3.0      # 작게 시작
    cam_angle = 0.0
    cam_height = 5.0
    # 필요하면 첫 도미노 자동으로 넘기기
    if dominoes.size() > 0:
        await get_tree().create_timer(0.8).timeout
        dominoes[0].apply_central_impulse(Vector3(2.5, 0, 0))
```

카메라 조작

```
func _process(delta):  
    if not cinematic_mode:  
        return  
  
    # 1. 반경 계속 증가 (점점 밖으로 빠져나감)  
    cam_radius += radius_speed * delta  
  
    # 2. 계속 회전  
    cam_angle += rotate_speed * delta  
  
    # 3. 높이도 천천히 올라감  
    cam_height += height_speed * delta  
  
    # 4. 카메라 위치 계산 (원점 기준으로 회전 + 확대)  
    var x = cos(cam_angle) * cam_radius  
    var z = sin(cam_angle) * cam_radius  
    var y = cam_height  
  
    camera.global_position = Vector3(x, y, z)  
  
    # 5. 항상 원점(나선 중심)을 바라보게  
    camera.look_at(Vector3(0, 3, 0), Vector3.UP)
```

파티클을 달면?



도미노에 부착

```
extends Rigidbody3D
```

```
@onready var particles: GPUParticles3D = $GPUParticles3D  
var has_fallen := false # bool은 := 로 초기화하는 게 최신 스타일
```

```
func _ready():  
    await get_tree().physics_frame
```

```
func _physics_process(_delta):  
    if has_fallen:  
        return  
  
    if global_position.y < 0.6:  
        has_fallen = true  
        emit_particles() # 여기서 바로 호출
```

```
func emit_particles():  
    if particles == null:  
        return # 안전장치
```

```
particles.restart() # One Shot  
particles.emitting = true # 자동으로 켜짐
```

```
get_tree().create_timer(1.5).timeout.connect(_on_particles_timeout)
```

```
func _on_particles_timeout():  
    if is_inside_tree() and particles:  
        particles.emitting = false
```